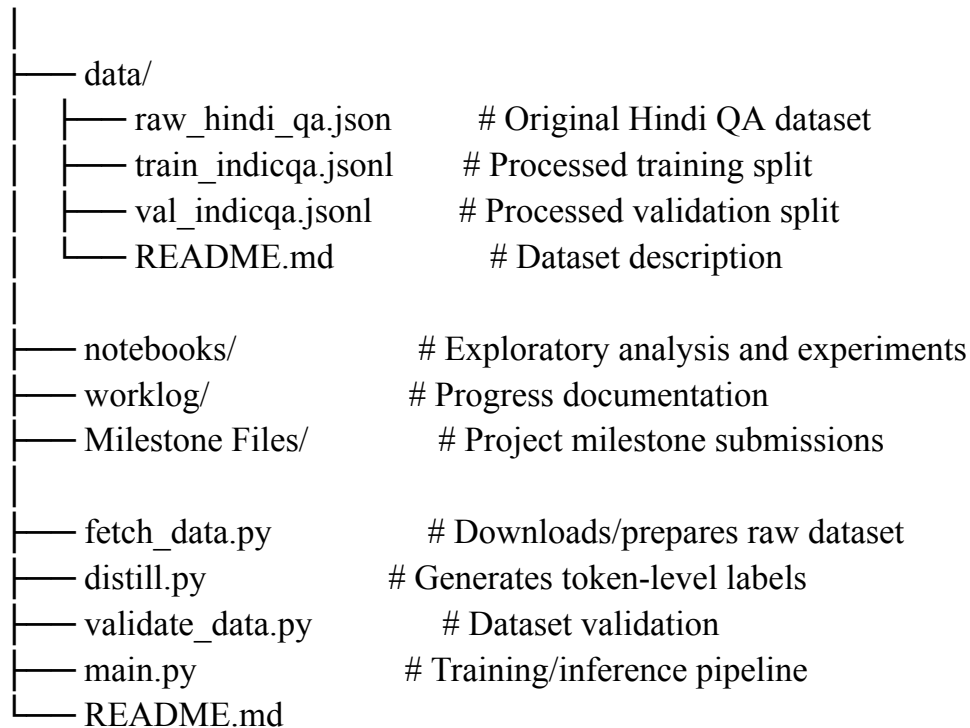


DSAI Project Milestone 3

Data Organization

The project maintains a clear separation between **raw data** and **processed data** to ensure reproducibility and simplify preprocessing and experimentation.

GROUP-9-DS-AND-AI-LAB-PROJECT/



Raw Data

The raw dataset is stored as: **data/raw_hindi_qa.json**

This file contains the original Hindi question-answer pairs before any preprocessing or token-level annotation.

Each sample consists of the original question, answer, and associated context.

Processed Data

After preprocessing and label generation, the dataset is converted into JSONL format suitable for training a token classification model.

data/train_indicqa.jsonl

data/val_indicqa.jsonl

- ❖ Each line represents one training instance containing:
- ❖ question – User query
- ❖ original_answer – Ground-truth answer
- ❖ tokens – Tokenized context passage
- ❖ labels – Binary labels indicating whether each token belongs to the compressed answer (1) or not (0)

Example:

Field	Value
Question	राजधानी दिल्ली में वाहनों की कुल कितनी संख्या है?
Original Answer	११२ लाख
Tokens	दिल्ली, के, सार्वजनिक, ..., ११२, लाख, वाहन, हैं
Labels	0, 0, 0, ..., 1, 1, 1, 0

Training And Validation Splits

Split	File	Purpose
Training	train_indicqa.jsonl	Used to train the token classification model
Validation	val_indicqa.jsonl	Used for hyperparameter tuning and performance evaluation during training
Test		

The training set is used to learn token-level compression labels, while the validation set measures the model's generalization performance and helps prevent overfitting.

Data Preprocessing

Before training the token classification model, the raw IndicQA Hindi dataset undergoes a series of preprocessing steps to transform it into a token-level labeled dataset suitable for supervised learning .

1. Data Acquisition

The preprocessing pipeline begins by downloading the Hindi subset of the IndicQA dataset from the Hugging Face repository. Specifically, the validation split stored in Parquet format is used.

For each record, the following fields are extracted:

- **Context:** Passage containing the answer.
- **Question:** User query.
- **Original Answer:** Ground-truth answer extracted from the **answers** field.

The extracted samples are stored in: data/raw_hindi_qa.json

{

```
"context": "...",  
"question": "...",  
"original_answer": "..."  
}
```

2. Context Distillation

The primary preprocessing step is **query-aware context compression**.

Each context is paired with its corresponding question and provided to the Qwen 3.5-397B language model through an API. A carefully designed system prompt instructs the model to:

- Remove all information irrelevant to the question.
- Preserve only the information necessary to answer the question.
- Maintain the original Hindi language.
- Avoid introducing new words or paraphrasing.
- Preserve grammatical correctness.

The output is therefore a compressed version of the original context containing only the most relevant information.

Original Context

दिल्ली भारत की राजधानी है... .. राष्ट्रीय राजधानी क्षेत्र में ११२ लाख वाहन हैं। ...

Compressed Context

राष्ट्रीय राजधानी क्षेत्र में ११२ लाख वाहन हैं।

This distilled context serves as the supervision signal for token labeling.

3. Tokenization

After context compression, both the original context and the compressed context are tokenized.

The current implementation performs **whitespace tokenization**, where text is split using spaces.

```
orig_tokens = original_text.split()  
comp_tokens = compressed_text.split()
```

For example,

Original context:

राष्ट्रीय राजधानी क्षेत्र में ११२ लाख वाहन हैं।

becomes

```
[  
  "राष्ट्रीय",  
  "राजधानी",  
  "क्षेत्र",  
  "में",  
  "११२",  
  "लाख",  
  "वाहन",  
  "हैं।"  
]
```

This tokenization strategy preserves the original word order while enabling token-level label generation.

4. Binary Label Generation

To train a token classification model, every token in the original context receives a binary label.

The compressed context tokens are converted into a set.

```
comp_tokens = set(compressed_text.split())
```

Each token from the original context is then compared against this set.

```
labels = [  
    1 if token in comp_tokens else 0  
    for token in orig_tokens  
]
```

where

- **1** indicates that the token is retained in the compressed context.
- **0** indicates that the token is discarded.

Token	Label
दिल्ली	0
भारत	0
राष्ट्रीय	1
राजधानी	1
क्षेत्र	1
११२	1

Thus, the preprocessing converts a span extraction problem into a sequence labeling problem.

5. Output Dataset Generation

Each processed sample is written as a single JSON object in JSON Lines (JSONL) format.

Each record contains:

```
{  
  "question": "...",  
  "original_answer": "...",  
  "tokens": [  
    "...",  
    "...",  
    "..."  
  ],  
  "labels": [  
    0,  
    0,  
    1,  
    1,  
    ...  
  ]  
}
```

The JSONL format allows efficient streaming during model training without loading the complete dataset into memory.

6. Incremental Processing

To support long-running preprocessing jobs and prevent data loss due to interruptions, the pipeline implements checkpointing.

Before processing begins, the output file is checked.

If previously processed samples exist,

- the number of completed records is counted,
- preprocessing resumes from the next unprocessed example,
- already processed samples are skipped.

This enables interrupted runs to continue without repeating earlier computations.

7. Error Handling and Retry Mechanism

Since context distillation relies on an external language model API, transient failures such as network timeouts or rate limiting may occur.

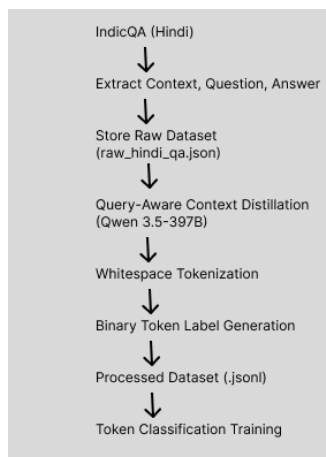
The preprocessing pipeline incorporates an automatic retry strategy:

- Maximum of **3 retries** per sample.
- Exponential backoff between retries.
- Permanent skipping of samples that continue to fail after all retry attempts.

This ensures robust preprocessing while minimizing interruptions caused by temporary API failures.

Final Processed Dataset

The preprocessing pipeline converts the raw question-answer dataset into a token-level sequence labeling dataset using the following workflow:

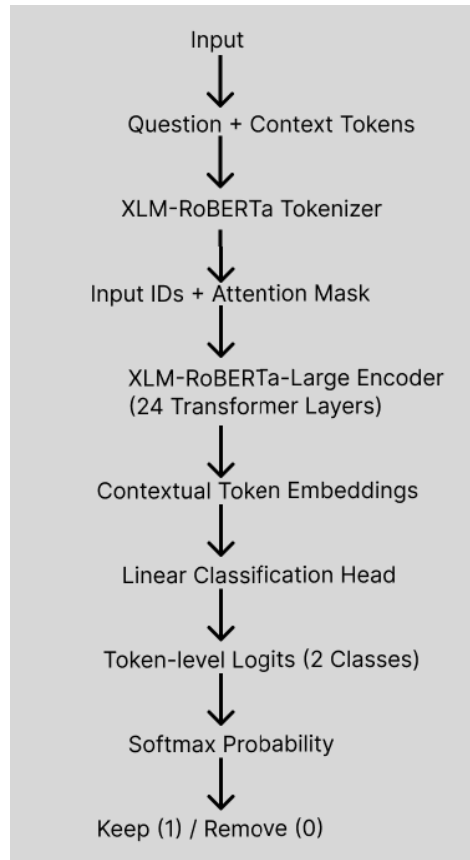


No text normalization (e.g., lowercasing, stop-word removal, stemming, or lemmatization) is applied because preserving the original Hindi text is essential for accurate token alignment and sequence labeling. Similarly, operations such as image resizing or pixel normalization are not applicable since the task is purely text-based.

Model Architecture

The proposed model is built upon XLM-RoBERTa-Large, a multilingual Transformer-based language model pretrained on over 100 languages using masked language modeling (MLM). In this work, XLM-RoBERTa-Large serves as the feature encoder, while a lightweight token classification head is fine-tuned to predict whether each token should be retained in the compressed context.

The overall architecture is illustrated below.



1. Input Representation

Each training example consists of:

- **Question**
- **Context passage**

The question and context are concatenated and tokenized using the XLM-RoBERTa tokenizer. The tokenizer converts the input text into:

- Input IDs
- Attention Mask

These inputs preserve multilingual text, including Hindi, without requiring language-specific preprocessing.

2. XLM-RoBERTa-Large Feature Encoder

The core of the architecture is the **XLM-RoBERTa-Large** encoder.

It contains:

- 24 Transformer encoder layers
- Hidden size of **1024**
- 16 self-attention heads per layer
- Approximately **560 million parameters**

Each Transformer layer consists of:

- Multi-head self-attention
- Feed-forward neural network
- Residual connections
- Layer normalization

The self-attention mechanism enables every token to attend to all other tokens in the sequence, allowing the model to capture long-range dependencies between the question and context.

The output of the encoder is a contextualized embedding for every input token.

3. Token Classification Head

The contextual embeddings generated by XLM-RoBERTa-Large are passed through a **linear classification layer**.

For every token, the classifier predicts one of two classes:

- **Class 0:** Token should be removed.
- **Class 1:** Token should be retained.

Mathematically,

$$z_i = Wh_i + b$$

where:

- h_i is the contextual embedding of the i_{th} token,
- W and b are the learnable parameters of the classification head,
- z_i represents the logits for the two output classes.

A softmax function converts these logits into probabilities, and the class with the highest probability is selected as the prediction.

4. Loss Function

The model is fine-tuned using a **token-level cross-entropy loss**.

During training:

- The predicted labels for each token are compared with the binary labels generated during preprocessing.
- Padding tokens are ignored when computing the loss.
- Gradients are backpropagated through both the classification head and the XLM-RoBERTa encoder, allowing the pretrained representations to adapt to the context compression task.

5. Model Output

For an input sequence of N tokens, the model produces:

$$Y = [y_1, y_2, y_3, \dots, y_N]$$

where

$$y_i \in \{0, 1\}$$

indicates whether the i^{th} token should be discarded (0) or retained (1).

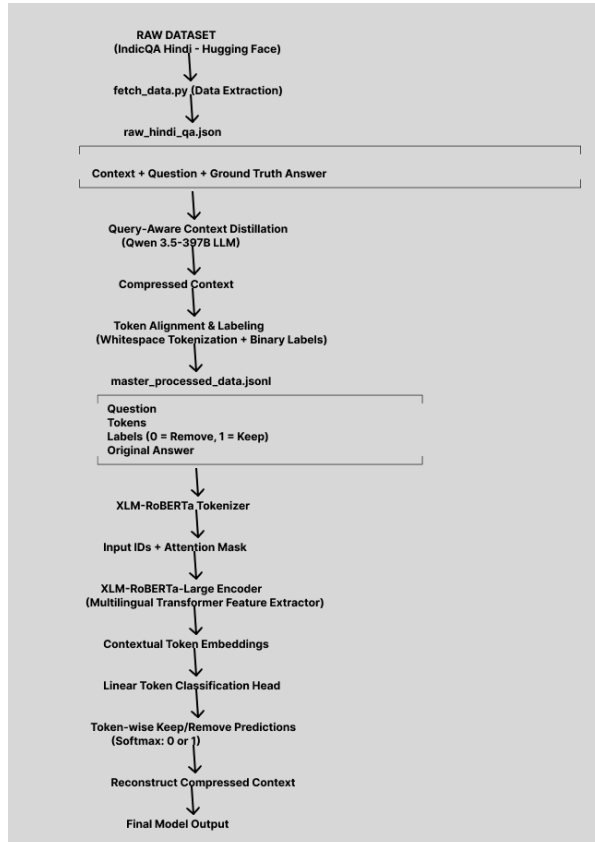
The retained tokens are combined in their original order to produce the compressed context while preserving the information required to answer the question.

Component Interaction

The interaction between the major components is summarized below:

1. **Tokenizer:** Converts the question-context pair into token IDs and attention masks compatible with XLM-RoBERTa.
2. **Feature Encoder (XLM-RoBERTa-Large):** Generates contextual embeddings by modeling semantic relationships between all tokens using self-attention.
3. **Classification Head:** Predicts a binary label for each token based on its contextual representation.
4. **Output Reconstruction:** Tokens predicted with label **1** are retained and concatenated to form the compressed context.

Overall DataFlow



Processed Data Format for Model Input

The preprocessing pipeline converts the raw Hindi question-answer dataset into a token-level sequence labeling dataset compatible with the **XLM-RoBERTa-Large** architecture. Each processed example consists of a question, tokenized context, and corresponding binary labels indicating whether each token should be retained in the compressed context.

The processed data is stored in JSONL format as follows:

```
{
  "question": "राजधानी दिल्ली में वाहनों की कुल कितनी संख्या है?",
  "original_answer": "११२ लाख",
  "tokens": [
    "दिल्ली",
    "के",
    "सार्वजनिक",
    "...",
    "११२",
    "लाख",
    "वाहन",
    "हैं।"
  ],
  "labels": [
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0
  ]
}
```

```

        "...",
        1,
        1,
        1,
        1
    ]
}

```

Each entry is subsequently converted into tensors required by XLM-RoBERTa-Large.

Input Construction

The model receives both the **question** and the **context** as input because token importance depends on the query.

The tokenizer constructs the sequence in the following format:

```
<s> Question </s></s> Context </s>
```

This enables the Transformer encoder to jointly model interactions between the question and context using self-attention.

Tokenization

The XLM-RoBERTa tokenizer performs subword tokenization using SentencePiece, converting the input text into multilingual subword units. Unlike the preprocessing stage, which uses whitespace tokenization for label generation, the model internally represents text as subword tokens.

The tokenizer produces three primary tensors:

- **Input IDs** – Integer indices representing vocabulary tokens.
- **Attention Mask** – Binary mask indicating valid tokens (1) and padding tokens (0).
- **Labels** – Binary token labels aligned with the tokenized sequence (non-context or special tokens are typically assigned an ignore index during training).

Input Tensor Format

After tokenization, each sample is converted into fixed-length tensors:

Tensor	Shape	Description
Input IDs	(B,L)	Token IDs from XLM-RobERta vocabulary
Attention Mask	(B,L)	Distinguishes Valid Token From Padding
Labels	(B,L)	Binary Labels (0,1)

For **batch size = 16** and **max sequence length = 512**:

- **Input IDs:** (16, 512)
- **Attention Mask:** (16, 512)
- **Labels:** (16, 512)

XLM-RoBERTa-Large supports sentences upto 512 tokens.

Embedding Representation

The embedding layer maps each token ID to a **1024-dimensional contextual vector**.

- Hidden size: **1024**
- Transformer layers: **24**
- Attention heads: **16**

Input tensors of shape **(B, L)** become contextual embeddings of shape:

(B, L, 1024)

Example:

(16, 512) → Embedding Layer → (16, 512, 1024)

Token Classification Output

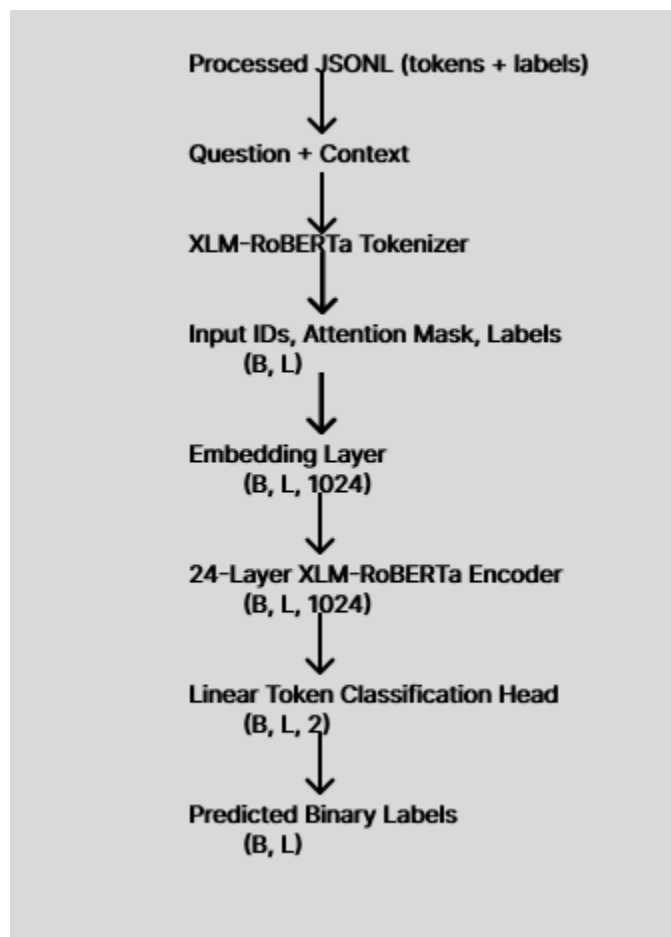
The contextual embeddings are passed through a linear classification head that predicts two classes for every token:

- 0: Remove
- 1: Keep

Output shapes:

- Logits: (B, L, 2)
- Predicted Labels: (B, L)

Training Data Flow



Why XLM-RoBERTa-Large

The proposed model uses **XLM-RoBERTa-Large** as the feature encoder with a **token classification head** to predict whether each token should be **kept (1)** or **removed (0)**. This architecture is well-suited for the task because it treats prompt compression as a

sequence labeling problem, enabling token-level decisions while preserving the original word order and semantic context.

XLM-RoBERTa-Large is pretrained on **100+ languages**, making it particularly effective for multilingual datasets such as **IndicQA**. Its deep Transformer architecture captures long-range dependencies between the **question** and **context**, allowing the model to identify the most informative tokens required to answer the question while discarding irrelevant ones. The token classification head directly produces binary labels for each token, making the model simple, efficient, and well-aligned with the objective of query-aware prompt compression.

Strengths

- **Strong multilingual representation**, enabling effective learning across multiple Indic languages.
- **Context-aware token selection**, leveraging self-attention to model relationships between all tokens.
- **Direct sequence labeling formulation**, eliminating the need for complex text generation or decoding.
- **Transfer learning benefits**, as pretrained knowledge reduces the amount of task-specific data required.
- **Scalable to multiple languages** without designing separate models for each language.

Limitations

- **Computationally expensive**, requiring significant GPU memory and training time due to the large model size.
- **Limited input length (512 tokens)**, requiring truncation or chunking for longer contexts.
- **Independent token predictions** may occasionally produce less coherent compressed sequences than generative models.
- **Higher inference latency** compared to lightweight Transformer variants such as DistilBERT or TinyBERT.

Comparison with Other Approaches

Compared to **extractive QA models**, the proposed architecture predicts token importance instead of answer spans, making it better suited for prompt compression. Unlike

generative encoder-decoder models (e.g., T5 or mT5), it avoids expensive autoregressive decoding, resulting in faster inference and simpler training. While lightweight models offer lower computational cost, they generally provide weaker contextual representations and lower multilingual performance. Overall, **XLM-RoBERTa-Large with a token classification head** provides an effective balance between multilingual understanding, contextual modeling, and token-level prediction accuracy, making it a strong choice for query-aware prompt compression.

Model Output

The model performs **binary token classification**, assigning each token one of two labels:

- 0 → Remove token
- 1 → Keep token

Example

Question: राजधानी दिल्ली में वाहनों की कुल कितनी संख्या है?

Original Context: राष्ट्रीय राजधानी क्षेत्र में ११२ लाख वाहन हैं। सन १९८५ में दिल्ली में प्रत्येक १००० व्यक्ति पर ८५ कारें थीं।

Predicted Labels: राष्ट्रीय → 1, राजधानी → 1, क्षेत्र → 1, में → 1, ११२ → 1, लाख → 1, वाहन → 1, हैं। → 1, सन → 0, १९८५ → 0, में → 0, दिल्ली → 0, ... → 0.

Reconstructed Compressed Context: राष्ट्रीय राजधानी क्षेत्र में ११२ लाख वाहन हैं।

Loss Function

The model is trained using **Cross-Entropy Loss**, which compares the predicted token labels with the ground-truth binary labels (0 or 1).

Evaluation Metrics

The model is evaluated using the **F1-Score**, which balances **precision** and **recall** and is well-suited for binary token classification, particularly when the classes are imbalanced.